**PRACTICAL 4 - STUDENT REGISTER**

Sepolia PoW Faucet : https://sepolia-faucet.pk910.de/?utm_source=chatgpt.com
Google Cloud Web3 Faucet - https://cloud.google.com/application/web3/faucet/ethereum/sepolia?utm_source=chatgpt.com

**CODE:**

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

// Contract to demonstrate Fallback and Receive
contract TestPayable {

    uint public x;
    uint public y;

    // Executes when Ether is received with empty calldata
    receive() external payable {
        x = 2;
        y = msg.value;
    }

    // Executes when unknown function data is received
    fallback() external payable {
        x = 1;
        y = msg.value;
    }
}


// Main Student Registration Contract
contract StudentRegister {

    // Mapping to store student records
    mapping(uint => Student) private students;

    address public owner;

    // Constructor
    constructor() payable {
        owner = msg.sender;
    }

    // Modifier to restrict registration to owner
    modifier onlyOwner() {
        require(msg.sender == owner, "Only owner can register");
```

```
    _;
}

// Student structure
struct Student {
    uint studentId;
    string name;
    uint[] marks;
    uint percentage;
    bool exist;
}

// Register a student
function register(
    uint studentId,
    string memory name,
    uint[] memory marks
) public onlyOwner returns (uint) {

    require(!students[studentId].exist,
        "Student data already exists");

    require(marks.length == 3,
        "Exactly 3 subjects are required");

    require(bytes(name).length > 0,
        "Name cannot be empty");

    uint totalMarks = getArraySum(marks);

    uint percentage = (totalMarks * 100) / 300;

    students[studentId] = Student(
        studentId,
        name,
        marks,
        percentage,
        true
    );

    return percentage;
}

// Get student details
function getStudentDetails(uint studentId)
    public
    view
    returns (
```

```
        uint,
        string memory,
        uint[] memory,
        uint
    )
{
    require(students[studentId].exist,
        "No student data available");

    Student memory student = students[studentId];

    return (
        student.studentId,
        student.name,
        student.marks,
        student.percentage
    );
}

// Calculate sum of marks
function getArraySum(uint[] memory array)
    private
    pure
    returns (uint sum)
{
    for (uint i = 0; i < array.length; i++) {

        require(array[i] <= 100,
            "Marks must be between 0 and 100");

        sum += array[i];
    }
}

// Call TestPayable contract to demonstrate fallback
function callTestPayable(TestPayable test)
    public
    payable
    returns (bool)
{
    // Call an unknown function with no Ether.
    // This triggers fallback().
    (bool success,) = address(test).call(
        abi.encodeWithSignature("nonExistingFunction()")
    );

    require(success, "Fallback call failed");
```

```
        // Call unknown function with 1 Wei.
        // This also triggers fallback().
        (success,) = address(test).call{value: 1}(
            abi.encodeWithSignature("nonExistingFunction()")
        );

        require(success, "Fallback payable call failed");

        // Send 2 Wei with empty calldata.
        // This triggers receive().
        (success,) = address(test).call{value: 2}("");

        require(success, "Receive call failed");

        return true;
    }
}
```

# Part 1 – Basic Solidity concepts

Before understanding the code, let's understand the first two lines.

```
None
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
```

1. SPDX License Identifier

- It specifies the license under which the code is shared.
- MIT is a commonly used open-source license.
- It helps avoid compiler license warnings.

2. Pragma Solidity

- It specifies the Solidity compiler version.
- `^0.8.20` means version 0.8.20 or a compatible newer version below 0.9.0.

# Part 2 – TestPayable Contract

Fallback and Receive

This contract is created to demonstrate how a smart contract handles Ether and unknown function calls.

### Code

```
None
contract TestPayable {

    uint public x;
    uint public y;

    receive() external payable {
        x = 2;
        y = msg.value;
    }

    fallback() external payable {
        x = 1;
        y = msg.value;
```

```
    }
}
```

### Line-by-line explanation

1. Contract declaration

```
None
contract TestPayable {
```

- `contract` is a keyword used to create a smart contract.
- `TestPayable` is the name of our contract.

2. Variables

```
None
uint public x;
uint public y;
```

- `uint` means unsigned integer. It stores only non-negative numbers.
- `public` means these variables can be accessed from outside the contract.
- `x` tells us which special function was executed.
- `y` stores the amount of Ether received, in Wei.

We made them public so Remix automatically provides buttons to check their values.

3. Receive function

```
None
receive() external payable {
    x = 2;
    y = msg.value;
}
```

Let's understand each keyword:

| Keyword | Meaning |
| --- | --- |
| `receive()` | Executes when Ether is received with empty calldata. |
| `external` | Allows the function to be called from outside the contract. |
| `payable` | Allows the function to receive Ether. |
| `msg.value` | Amount of Ether sent with the current call, in Wei. |

Inside the function:

- `x = 2` means receive function has executed.
- `y = msg.value` stores the amount received.

Example: If someone sends 2 Wei to this contract with empty calldata:

- x becomes 2.
- y becomes 2.

4. Fallback function

```
None
fallback() external payable {
    x = 1;
    y = msg.value;
}
```

Fallback is a special function that executes when someone calls a function that does not exist in the contract.

For example, suppose someone calls `hello()` but our contract does not contain a `hello()` function.

Instead of immediately failing, Solidity executes the fallback function.

Here:

- `external` allows external calls.
- `payable` allows it to receive Ether.
- `x = 1` indicates fallback was executed.
- `y = msg.value` stores the received amount.

### Difference between receive and fallback

| receive() | fallback() |
| --- | --- |
| Executes when calldata is empty and Ether is received. | Executes when the function call does not match an existing function. |
| Must be payable. | Can be payable or non-payable. |
| Used for direct Ether transfers. | Used for unknown function calls and certain unmatched calls. |

Remember: If calldata is empty and `receive()` exists, Solidity uses `receive()`. Otherwise, `fallback()` may handle the call.

# Part 3 – StudentRegister Contract

Main Student Management Contract

This is our main contract. It stores student information and allows us to retrieve it.

### 1. Contract declaration

```
None
contract StudentRegister {
```

We created a smart contract named `StudentRegister` to manage student records.

### 2. Mapping

```
None
mapping(uint => Student) private students;
```

First, understand mapping.

A mapping stores data in the form of key-value pairs.

Here:

- `uint` is the key, which represents student ID.
- `Student` is the value, which represents complete student information.
- `students` is the mapping name.
- `private` means it can be accessed directly only inside this contract.

Example:

| Student ID (Key) | Student information (Value) |
|---|---|
| 101 | Ruhi, [80,90,70], 80% |
| 102 | Rahul, [70,80,90], 80% |

Why did we use mapping?

Because every student has a unique ID, and we can use that ID to find their information.

### 3. Owner variable

```
None
address public owner;
```

- `address` is a Solidity data type used to store an Ethereum account or contract address.
- `owner` stores the address of the person who deployed the contract.
- `public` allows us to check the owner address from Remix.

### 4. Constructor

```
None
constructor() payable {
    owner = msg.sender;
}
```

A constructor is a special function that executes automatically when the contract is deployed.

Here:

- `constructor()` is called only during deployment.
- `payable` allows the constructor to receive Ether during deployment.
- `msg.sender` represents the address that deployed the contract.
- `owner = msg.sender` saves that address as the owner.

Example: If your MetaMask account deploys the contract, your wallet address becomes the owner.

### 5. Modifier

```
None
modifier onlyOwner() {
    require(msg.sender == owner, "Only owner can register");
    _;
}
```

A modifier is used to apply a condition before executing a function.

Here:

- `modifier onlyOwner()` creates a modifier named `onlyOwner`.
- `require()` checks whether the caller is the owner.
- If the condition is false, the transaction is reverted.
- `_` represents the place where the original function's code executes.

Simple example:

Only the person who deployed the contract can register student details.

If another person tries to register a student, the transaction will fail.

### 6. Structure

```
None
struct Student {
```

```
    uint studentId;
    string name;
    uint[] marks;
    uint percentage;
    bool exist;
}
```

A structure is a user-defined data type that groups multiple variables of different data types under one name.

Here we created a structure called `Student`.

| Variable | Data type | Purpose |
|---|---|---|
| studentId | uint | Stores student ID |
| name | string | Stores student name |
| marks | uint[] | Stores subject marks |
| percentage | uint | Stores percentage |
| exist | bool | Checks whether student exists |

What is `uint[]`?

It is a dynamic array of unsigned integers.

For example:

```
None
[80, 90, 70]
```

It stores marks for three subjects.

What is `bool`?

Boolean stores either `true` or `false`.

Here, `exist` tells us whether the student record is already registered.

# Part 4 – Register Function

This function is used to register a new student.

### Code

```
None
function register(
    uint studentId,
    string memory name,
    uint[] memory marks
) public onlyOwner returns (uint) {
```

Let's understand the declaration:

- `function` creates a function.
- `register` is the function name.
- `uint studentId` accepts the student ID.
- `string memory name` accepts the student name.
- `uint[] memory marks` accepts an array of marks.
- `public` allows the function to be called from outside the contract.
- `onlyOwner` ensures only the owner can register students.
- `returns (uint)` means the function returns an unsigned integer.

What is memory?

`memory` is temporary storage used while a function executes. Here, the name and marks are temporarily available during the function call.

### 1. Check duplicate student ID

```
None
require(!students[studentId].exist,
    "Student data already exists");
```

This checks whether the student ID is already registered.

- `students[studentId].exist` checks the existence value.
- `!` means NOT.
- If the student already exists, registration is rejected.

### 2. Check number of subjects

```
None
require(marks.length == 3,
    "Exactly 3 subjects are required");
```

This ensures that exactly three marks are entered.

For example:

- `[80,90,70]` – Valid.
- `[80,90]` – Invalid.
- `[80,90,70,60]` – Invalid.

### 3. Check student name

```
None
require(bytes(name).length > 0,
    "Name cannot be empty");
```

This checks that the student name is not empty.

### 4. Calculate total marks

```
None
uint totalMarks = getArraySum(marks);
```

This calls the `getArraySum()` function to calculate the sum of all marks.

For example:

80 + 90 + 70 = 240.

### 5. Calculate percentage

```
None
uint percentage = (totalMarks * 100) / 300;
```

Maximum marks are 300 because there are three subjects, each having a maximum of 100 marks.

For our example:

$$\frac{240 \times 100}{300}=80\%$$

Therefore, percentage = 80.

### 6. Store student information

```
None
students[studentId] = Student(
    studentId,
    name,
    marks,
    percentage,
    true
);
```

This creates a Student structure and stores all the information in the mapping.

The final `true` indicates that the student has been registered.

### 7. Return percentage

```
None
return percentage;
```

This returns the calculated percentage.

Complete working of register():

Input → Validate student details → Calculate total marks → Calculate percentage → Store student information → Return percentage.

# Part 5 – Get Student Details Function

```
None
function getStudentDetails(uint studentId)
    public
    view
    returns (
```

```
        uint,
        string memory,
        uint[] memory,
        uint
    )
```

This function retrieves the details of a registered student.

Important keywords:

- `public`: Can be called from outside the contract.
- `view`: Reads stored information but does not modify it.
- `returns`: Specifies the values returned by the function.

### 1. Check whether student exists

```
None
require(students[studentId].exist,
    "No student data available");
```

If the student does not exist, it displays an error.

### 2. Copy student information

```
None
Student memory student = students[studentId];
```

This retrieves the student record from the mapping and stores a temporary copy in memory.

### 3. Return student information

```
None
return (
    student.studentId,
    student.name,
    student.marks,
    student.percentage
);
```

It returns the student's ID, name, marks and percentage.

# Part 6 – Get Array Sum Function

```
None
function getArraySum(uint[] memory array)
    private
    pure
    returns (uint sum)
```

This function calculates the total marks.

Understand the keywords:

- `private`: Can be called only inside the same contract.
- `pure`: Does not read or modify stored blockchain state.
- `returns (uint sum)`: Returns an unsigned integer named `sum`.

### Loop

```
None
for (uint i = 0; i < array.length; i++) {
```

A `for` loop is used to visit every element of the array.

- `uint i = 0`: Starts from index 0.
- `i < array.length`: Continues until all elements are visited.
- `i++`: Increases i by 1 after every iteration.

### Validate marks

```
None
require(array[i] <= 100,
    "Marks must be between 0 and 100");
```

Checks that every subject mark is less than or equal to 100.

### Add marks

```
None
sum += array[i];
```

Adds each mark to the total sum.

This is equivalent to:

```
None
sum = sum + array[i];
```

### Dry run

For marks `[80,90,70]`:

| Iteration | i | array[i] | sum |
|---|---|---|---|
| 1 | 0 | 80 | 80 |
| 2 | 1 | 90 | 170 |
| 3 | 2 | 70 | 240 |

Final sum = 240.

# Part 7 – Call TestPayable Function

Most important for Fallback practical

This function connects our two contracts and tests the fallback and receive functions.

### Function declaration

```
None
function callTestPayable(TestPayable test)
    public
    payable
    returns (bool)
```

Let's understand:

- `callTestPayable` is the function name.
- `TestPayable test` accepts the address of the deployed TestPayable contract.
- `public` means it can be called from outside.

- `payable` allows this function to receive Ether.
- `returns (bool)` returns either true or false.

### First call – Fallback without Ether

```
None
(bool success,) = address(test).call(
    abi.encodeWithSignature("nonExistingFunction()")
);
```

This line calls a function named `nonExistingFunction()` on the TestPayable contract.

But that function does not exist.

Therefore, the `fallback()` function executes.

- `address(test)` gets the address of the TestPayable contract.
- `.call()` makes a low-level external call.
- `abi.encodeWithSignature()` prepares the function call data.
- `success` stores whether the call succeeded.
- The comma ignores the returned data.

```
None
require(success, "Fallback call failed");
```

Checks whether the call was successful. If not, the transaction is reverted.

### Second call – Fallback with 1 Wei

```
None
(success,) = address(test).call{value: 1}(
    abi.encodeWithSignature("nonExistingFunction()")
);
```

This again calls a function that does not exist.

But this time, it sends 1 Wei along with the call.

Therefore, fallback executes and receives 1 Wei.

### Third call – Receive with 2 Wei

```
None
(success,) = address(test).call{value: 2}("");
```

This sends 2 Wei with empty calldata.

Since the calldata is empty, the `receive()` function executes.

### Return statement

```
None
return true;
```

If all three calls execute successfully, the function returns `true`.

## Why do we enter 3 Wei in Remix?

Our function forwards:

- First call: 0 Wei.
- Second call: 1 Wei.
- Third call: 2 Wei.

Total = 0 + 1 + 2 = 3 Wei.

Therefore, we enter 3 Wei in the Value field before calling `callTestPayable()`.

If we enter 0 Wei, the StudentRegister contract will not have enough Ether to forward the required 3 Wei, so the transaction will revert.

## Dry run of fallback and receive

| Step | Function executed | Ether received | x | y |
|---|---|---|---|---|
| Initial | None | 0 | 0 | 0 |
| First call | fallback() | 0 Wei | 1 | 0 |

| | | | | |
|---|---|---|---|---|
| Second call | fallback() | 1 Wei | 1 | 1 |
| Third call | receive() | 2 Wei | 2 | 2 |

Final output

- `x = 2`
- `y = 2`

Why does x become 2?

Because the last function executed was `receive()`, which changed x from 1 to 2.

# Part 8 – Important Solidity keywords revision

| Keyword | Simple meaning |
|---|---|
| `contract` | Creates a smart contract |
| `uint` | Stores non-negative integers |
| `string` | Stores text |
| `bool` | Stores true or false |
| `address` | Stores a blockchain address |
| `mapping` | Stores key-value pairs |

| | |
|---|---|
| `struct` | Groups different variables |
| `public` | Accessible from inside and outside |
| `private` | Accessible only inside the same contract |
| `external` | Called from outside the contract |
| `memory` | Temporary data storage |
| `payable` | Allows receiving Ether |
| `view` | Reads data without changing it |
| `pure` | Does not read or change contract state |
| `require` | Checks a condition |
| `return` | Sends a result back |
| `msg.sender` | Address of the caller |
| `msg.value` | Ether sent with the current call |

# Part 9 – Complete practical dry run

Let's understand the entire program with one example.

Step 1: Deploy `TestPayable`.

Step 2: Deploy `StudentRegister`. The wallet that deploys it becomes the owner.

Step 3: Register a student.

Input:

```
None
studentId = 101
name = "Ruhi"
marks = [80, 90, 70]
```

Output:

```
None
Student ID = 101
Name = Ruhi
Marks = [80, 90, 70]
Percentage = 80
```

Step 4: Call `getStudentDetails(101)` to retrieve the student details.

Step 5: Call `callTestPayable()` with the deployed TestPayable address and Value = 3 Wei.

Step 6: Check the values:

```
None
x() = 2
y() = 2
```

Step 7: Open the transaction on Sepolia Etherscan and observe gas used, gas price and transaction fee.

# Part 10 – Important viva questions

| Question | Answer |
| --- | --- |
| What is Solidity? | A programming language used to write smart contracts. |
| What is a smart contract? | A program deployed on a blockchain that executes predefined instructions. |
| What is a structure? | A user-defined data type that groups different variables. |
| What is an array? | A collection of elements of the same data type. |
| What is mapping? | A key-value pair data structure. |
| What is a constructor? | A special function executed once during deployment. |
| What is a modifier? | A reusable condition applied to a function. |
| What is fallback? | A function executed when a call does not match an existing function. |
| What is receive? | A function executed when Ether is received with empty calldata. |
| What is payable? | A keyword that allows a function to receive Ether. |
| What is msg.sender? | Address of the caller. |
| What is msg.value? | Amount of Ether sent with the call, in Wei. |

| | |
|---|---|
| What is Wei? | The smallest unit of Ether. |
| What is gas? | A measure of computational work on Ethereum. |
| Why did we use 300? | Maximum marks are 100 for each of three subjects. |
| Why did we use 3 Wei? | The function forwards 1 Wei and 2 Wei in its second and third calls. |

# BCT Practical 4 – Student Data Using Solidity

**Aim:** Write and deploy a Solidity smart contract to create and manage student data using Structures, Arrays and Fallback. Observe transaction fees and gas values.

## 1. Requirements

- MetaMask browser extension
- Remix IDE: https://remix.ethereum.org/
- Sepolia Test Network enabled in MetaMask
- Some Sepolia test ETH (not real ETH)

**Important:** Never share your MetaMask recovery phrase or private key.

## 2. Write the Solidity program

1. Open Remix IDE.
2. Go to the File Explorer.
3. Create a new file named `student.sol`.
4. Paste the following code:

None

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract TestPayable {
    uint public x;
    uint public y;

    // Executes when Ether is sent with empty calldata
    receive() external payable {
        x = 2;
        y = msg.value;
    }

    // Executes when an unknown function is called
    fallback() external payable {
        x = 1;
        y = msg.value;
    }
}

contract StudentRegister {
    mapping(uint => Student) private students;
    address public owner;

    constructor() payable {
        owner = msg.sender;
    }

    modifier onlyOwner() {
        require(msg.sender == owner, "Only owner can register");
        _;
    }

    // Structure to store student information
    struct Student {
        uint studentId;
        string name;
        uint[] marks;
        uint percentage;
```

```
    bool exist;
}

// Register student details
function register(
    uint studentId,
    string memory name,
    uint[] memory marks
) public onlyOwner returns (uint) {

    require(!students[studentId].exist,
        "Student data already exists");

    require(marks.length == 3,
        "Exactly 3 subjects are required");

    require(bytes(name).length > 0,
        "Name cannot be empty");

    uint totalMarks = getArraySum(marks);
    uint percentage = (totalMarks * 100) / 300;

    students[studentId] = Student(
        studentId,
        name,
        marks,
        percentage,
        true
    );

    return percentage;
}

// Retrieve student details
function getStudentDetails(uint studentId)
    public view
    returns (uint, string memory, uint[] memory, uint)
{
    require(students[studentId].exist,
```

```
        "No student data available");

    Student memory student = students[studentId];

    return (
        student.studentId,
        student.name,
        student.marks,
        student.percentage
    );
}

// Calculate total marks
function getArraySum(uint[] memory array)
    private pure returns (uint sum)
{
    for (uint i = 0; i < array.length; i++) {
        require(array[i] <= 100,
            "Marks must be between 0 and 100");

        sum += array[i];
    }
}

// Test fallback and receive functions
function callTestPayable(TestPayable test)
    public payable returns (bool)
{
    (bool success,) = address(test).call(
        abi.encodeWithSignature("nonExistingFunction()")
    );

    require(success, "Fallback call failed");

    (success,) = address(test).call{value: 1}(
        abi.encodeWithSignature("nonExistingFunction()")
    );

    require(success, "Fallback payable call failed");
```

```
        (success,) = address(test).call{value: 2}("");

        require(success, "Receive call failed");

        return true;
    }
}
```

**Note:** This is the updated version we used during testing. It includes both `receive()` and `fallback()`. The original practical PDF's `TestPayable` code only contained `receive()`.

# 3. Compile the program

1. Open the Solidity Compiler tab.
2. Select compiler version `0.8.20`.
3. Select EVM version `Paris`.
4. Click **Compile student.sol**.
5. Check that compilation is successful.

# 4. Connect MetaMask

1. Open MetaMask.
2. Select the **Sepolia Test Network**.
3. Open Remix → Deploy & Run Transactions.
4. Select Environment: **Remix Browser Extension**.
5. Connect and approve the MetaMask connection.
6. Make sure the selected account is your Sepolia account.

# 5. Deploy the contracts

The Solidity file contains two contracts, so deploy both separately.

**First – Deploy TestPayable**

1. Select `TestPayable` from the Contract dropdown.
2. Keep Value as `0`.
3. Click Deploy.
4. Confirm the transaction in MetaMask.
5. Wait for deployment success.

**Second – Deploy StudentRegister**

1. Select `StudentRegister` from the Contract dropdown.
2. Keep Value as `0`.
3. Click Deploy.
4. Confirm the transaction in MetaMask.
5. Wait for deployment success.

Both contracts should now appear under Deployed Contracts.

# 6. Register student details

1. Expand the latest deployed `StudentRegister` contract.
2. Find the `register` function.
3. Enter the following sample values:

| Parameter | Value |
|---|---|
| studentId | 101 |
| name | "Ruhi" |
| marks | [80,90,70] |

4. Click the orange `register` button.
5. Confirm the transaction in MetaMask.
6. Wait for successful execution.

**Percentage calculation:**

Total marks = 80 + 90 + 70 = 240

Maximum marks = 300

Percentage = (240 / 300) × 100 = **80%**

# 7. Retrieve student details

1. Find `getStudentDetails`.
2. Enter student ID `101`.
3. Click the blue button.
4. You should get:

- Student ID: 101
- Name: Ruhi
- Marks: 80, 90, 70
- Percentage: 80

This is a view function, so it does not require a paid on-chain transaction.

# 8. Test Fallback and Receive

1. Expand the deployed `StudentRegister` contract.
2. Find `callTestPayable`.
3. Enter the address of your deployed `TestPayable` contract.
4. In the Remix Value field, select **Wei**.
5. Enter `3`.
6. Click `callTestPayable`.
7. Confirm the transaction in MetaMask.

The function performs three calls:

| Call | Calldata | Ether sent | Function executed |
|---|---|---|---|
| 1 | Unknown function | 0 Wei | fallback() |
| 2 | Unknown function | 1 Wei | fallback() |
| 3 | Empty calldata | 2 Wei | receive() |

The transaction should complete successfully.

**Verify the final values:**

Expand `TestPayable` and click:

- `x()` → Expected output: 2
- `y()` → Expected output: 2 Wei

The final values are from the last `receive()` call.

# 9. Observe Gas and Transaction Fee

After every successful state-changing transaction:

1. Open the transaction details in Remix Terminal.
2. Find the transaction hash.
3. Open the hash on Sepolia Etherscan.
4. Observe Gas Used, Gas Price and Transaction Fee.

Etherscan: https://sepolia.etherscan.io/

**Formula:**

Transaction Fee = Gas Used × Gas Price

Remember:

- Gas Used: Actual gas consumed by the transaction.
- Gas Price: Cost per unit of gas.
- Gas Limit: Maximum gas allowed for the transaction.
- Transaction Fee: Total fee paid for the transaction.

### Our observed transaction

For the successful `callTestPayable()` transaction:

| Parameter | Observed value |
|---|---|
| Network | Sepolia Testnet |
| Gas Used | 265,443 |
| Gas Price | 1.500000014 Gwei |
| Transaction Fee | 0.000398164503716202 ETH |
| Transaction Value | 22 Wei |
| Status | Success |

These are the values from our previous test transaction. Your friend's or your next transaction may have different gas values.

## 10. Result

The StudentRegister smart contract was successfully compiled and deployed on the Ethereum Sepolia Test Network. Student details were stored and retrieved using structures and arrays. The fallback and receive functions were tested successfully, and transaction gas usage and fees were observed.

---

# Important Viva Questions

**1. What is a smart contract?**

A smart contract is a self-executing program deployed on a blockchain. It runs when specified conditions are met.

**2. What is Solidity?**

Solidity is a high-level programming language used to write smart contracts for Ethereum-compatible blockchains.

**3. What is a structure?**

A structure is a user-defined data type that groups different variables under one name.

Example: `Student` stores student ID, name, marks and percentage.

**4. What is an array?**

An array stores multiple values of the same data type. Here, `marks` stores marks of three subjects.

**5. What is a mapping?**

A mapping stores key-value pairs. Here, student ID is the key and student information is the value.

**6. What is fallback()?**

Fallback is a special function executed when a contract receives a call that does not match any existing function. It can be payable to accept Ether.

**7. What is receive()?**

Receive is a special function executed when a contract receives Ether with empty calldata.

**8. Difference between fallback() and receive()?**

- `receive()` executes when calldata is empty.
- `fallback()` executes when the function call does not match an existing function, or when calldata is empty and no receive function exists.

**9. What is msg.sender?**

It represents the address of the account or contract that called the current function.

**10. What is msg.value?**

It represents the amount of Ether sent with the current call, in Wei.

**11. What is Wei?**

Wei is the smallest unit of Ether. 1 ETH = $10^{18}$ Wei.

**12. What is gas in Ethereum?**

Gas measures the computational work required to execute operations on the Ethereum network.

**13. Why do we use payable?**

The `payable` keyword allows a function or address to receive Ether.

**14. What is a modifier?**

A modifier adds conditions to a function. Here, `onlyOwner` ensures that only the contract owner can register students.

**15. What is the difference between view and pure?**

- `view` reads blockchain state but does not modify it.
- `pure` neither reads nor modifies blockchain state.

**16. Why is percentage divided by 300?**

There are three subjects, each having a maximum of 100 marks. Therefore, total maximum marks are 300.